Machine Learning Intrusion Detection System (IDS)

Internship project demo — Google Colab

This notebook uses synthetic educational network-traffic data to demonstrate training and evaluating a classifier. It is not a real-world security detector.

Open this file in Google Colab, then choose **Runtime → Run all**.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, ConfusionMatrixDisplay
from IPython.display import display
print('Libraries loaded successfully.')
```

```
Libraries loaded successfully.
```

1. Generate synthetic educational traffic data

```
rng = np.random.default_rng(42)
n = 2000
duration = rng.exponential(scale=3.0, size=n)
bytes_sent = rng.lognormal(mean=7.0, sigma=1.0, size=n)
packets = rng.poisson(lam=25, size=n) + 1
failed_connections = rng.poisson(lam=0.5, size=n)
unique_ports = rng.integers(1, 8, size=n)
risk = failed_connections * 1.5 + unique_ports * 0.8 + (packets > 55) * 1.2 + (bytes_sent > 5000) * 0.7
labels = (risk > 5.2).astype(int)
flip = rng.random(n) < 0.04
labels[flip] = 1 - labels[flip]
data = pd.DataFrame({'duration': duration.round(3), 'bytes_sent': bytes_sent.round(2), 'packets': packets, 'failed_connectio
data['label_name'] = data['label'].map({0:'Normal', 1:'Suspicious'})
print('Dataset shape:', data.shape)
display(data.head())
display(data['label_name'].value_counts().to_frame('Rows'))
```

```
Dataset shape: (2000, 7)
```

	duration	bytes_sent	packets	failed_connections	unique_ports	label	label_name
0	7.213	289.03	23	1	2	0	Normal
1	7.009	251.69	37	0	4	0	Normal
2	7.154	1929.76	33	0	5	0	Normal
3	0.839	81.30	26	1	6	1	Suspicious
4	0.259	1346.70	30	1	3	0	Normal

label_name	Rows
Normal	1388
Suspicious	612

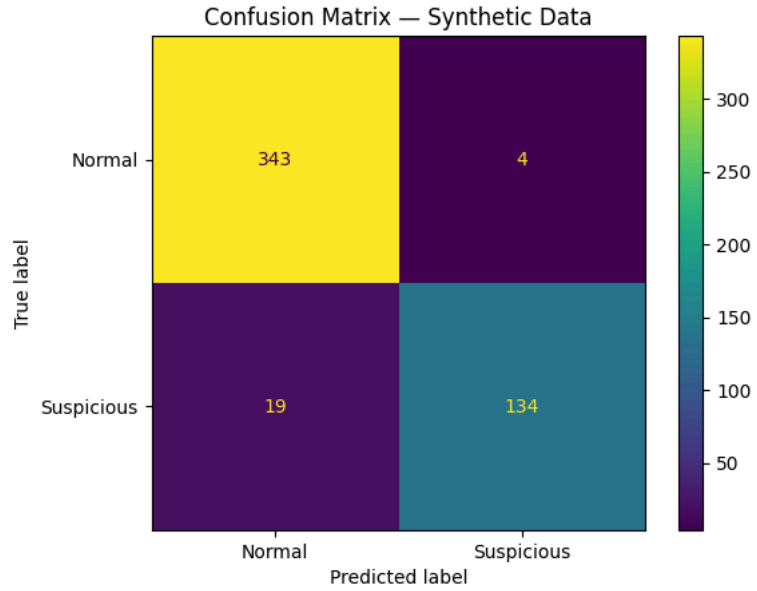
2. Train and evaluate a Random Forest classifier

```
features = ['duration', 'bytes_sent', 'packets', 'failed_connections', 'unique_ports']
X_train, X_test, y_train, y_test = train_test_split(data[features], data['label'], test_size=0.25, random_state=42, stratif
model = RandomForestClassifier(n_estimators=150, random_state=42, class_weight='balanced')
model.fit(X_train, y_train)
pred = model.predict(X_test)
print(f'Accuracy: {accuracy_score(y_test, pred):.3f}')
print(classification_report(y_test, pred, target_names=['Normal', 'Suspicious'], zero_division=0))
ConfusionMatrixDisplay.from_predictions(y_test, pred, display_labels=['Normal', 'Suspicious'])
plt.title('Confusion Matrix — Synthetic Data')
plt.show()
```

```
Accuracy: 0.954
              precision    recall  f1-score   support

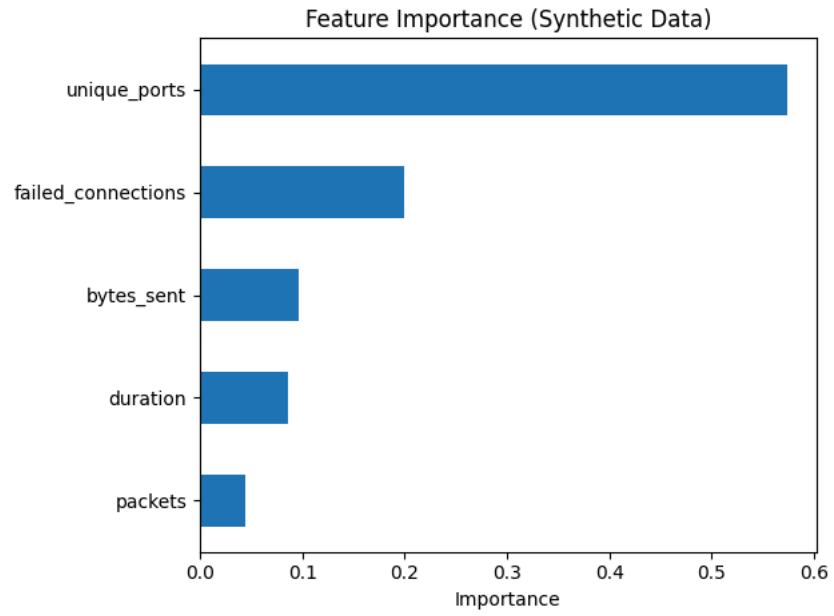
      Normal       0.95      0.99      0.97       347
  Suspicious       0.97      0.88      0.92       153

    accuracy                           0.95       500
   macro avg       0.96      0.93      0.94       500
weighted avg       0.95      0.95      0.95       500
```



3. Feature importance

```
pd.Series(model.feature_importances_, index=features).sort_values().plot(kind='barh', title='Feature Importance (Synthetic
plt.xlabel('Importance')
plt.tight_layout()
plt.show()
```



4. Optional: upload a CSV and predict

CSV must include numeric columns: `duration`, `bytes_sent`, `packets`, `failed_connections`, `unique_ports`. Only analyze data you are authorized to use.

```
from google.colab import files
uploaded = files.upload()
if uploaded:
    filename = next(iter(uploaded))
```

```
    custom = pd.read_csv(filename)
    missing = [c for c in features if c not in custom.columns]
    if missing:
        print('Missing required columns:', missing)
    else:
        custom['prediction'] = model.predict(custom[features])
        custom['prediction_name'] = custom['prediction'].map({0:'Normal', 1:'Suspicious'})
        display(custom.head(20))
        custom.to_csv('ids_predictions.csv', index=False)
        files.download('ids_predictions.csv')
```

Choose Files No file chosen

5. Export the trained model

```
import joblib
joblib.dump({'model': model, 'features': features}, 'ids_model.joblib')
print('Saved ids_model.joblib')
files.download('ids_model.joblib')
```

```
Saved ids_model.joblib
```

Report notes and limitations

- **Goal:** demonstrate an ML workflow for classifying illustrative traffic records.
- **Algorithm:** Random Forest.
- **Features:** duration, bytes sent, packet count, failed connections, and unique ports.
- **Evaluation:** accuracy, precision, recall, F1-score, and confusion matrix.
- **Important limitation:** labels are generated from synthetic rules. Good results here do not prove effectiveness on real traffic. A real IDS requires suitable, legally obtained data and careful testing for false positives and false negatives.